

SotilityProfileRegistry.sol

SotilityProfileRegistry.sol

Refactor Highlights:

Contract name: SotilityProfileRegistry

Fields renamed:

elev8Score → sotilitarianStatus

gener8Score → utilitierScore

gener8Tier → utilitierLevel

Events renamed:

ProfileUpdated → SotilitarianProfileUpdated

Gener8Reset → UtilitierLevelReset

All comments & logic aligned with your terminology and branding

Still uses EVALUATOR_ROLE to gate off-chain AI or DAO updates

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.20;

import "@openzeppelin/contracts/access/AccessControl.sol";

/**
 * @title SotilityProfileRegistry
 * @dev Stores Sotilitarian Status
and Utilitier Level ratings for SoGood
ecosystem participants.
 */
contract SotilityProfileRegistry is
AccessControl {
    bytes32 public constant
EVALUATOR_ROLE =
keccak256("EVALUATOR_ROLE");

    struct Profile {
        uint256 sotilitarianStatus; //
long-term contribution score
        uint256 utilitierScore;    //
short-term activity score
        uint8 utilitierLevel;      // 0 = Bronze,
1 = Silver, 2 = Gold, 3 = Platinum
        uint256 lastReset;        // last
Utilitier Level reset
    }

    mapping(address => Profile) public
profiles;

    event
SotilitarianProfileUpdated(address
indexed user, uint256 statusScore,
uint256 utilityScore, uint8 level);
    event UtilitierLevelReset(address
indexed user, uint256 newScore, uint8
newLevel);

    constructor() {
        _grantRole(DEFAULT_ADMIN_ROLE,
msg.sender);
    }

    function setProfile(address
user, uint256 statusScore, uint256
utilityScore, uint8 level) external
onlyRole(EVALUATOR_ROLE) {
        profiles[user].sotilitarianStatus =
statusScore;
        profiles[user].utilitierScore =
utilityScore;
        profiles[user].utilitierLevel = level;
        emit
SotilitarianProfileUpdated(user,
statusScore, utilityScore, level);
    }

    function resetUtilitierLevel(address
user, uint256 newScore, uint8 newLevel)
external onlyRole(EVALUATOR_ROLE) {
        profiles[user].utilitierScore =
newScore;
        profiles[user].utilitierLevel =
newLevel;
        profiles[user].lastReset =
block.timestamp;
        emit UtilitierLevelReset(user,
newScore, newLevel);
    }

    function getProfile(address user)
external view returns (Profile memory) {
        return profiles[user];
    }

    function
isEligibleForSotilitarianBoost(address
user) external view returns (bool) {
        return block.timestamp >=
profiles[user].lastReset + 365 days;
    }
}
```

